

Sisteme Client-Server și Execuție la Distanță în Python

Paul A. Gagniuc

Cuprins

- Ce este un endpoint HTTP.
- Ce este un REST API.
- Comunicarea client-server.
- Structura unei cereri HTTP.
- Trimiterea datelor prin metoda POST.
- Comunicarea prin IP și port.
- Utilizarea serverului local XAMPP.
- Rolul directorului `htdocs`.
- Scrierea unui client HTTP în Python.
- Utilizarea `urllib.request` și `urllib.parse`.
- Codificarea parametrilor HTTP.
- Recepția și procesarea răspunsului serverului.
- Construirea unui server PHP simplu.
- Salvarea datelor pe server.
- Identificarea clientului prin IP.
- Gestionarea mesajelor text.
- Construirea unei interfețe server live în browser.
- Actualizarea automată a paginii web.
- Capturarea traficului de rețea cu Wireshark.
- Vizualizarea datelor în clar în rețea.
- Necesitatea criptării comunicațiilor.
- Modificarea endpoint-urilor fără schimbarea logicii clientului.
- Injectarea și transmiterea de valori numerice.
- Generarea de trenduri grafice.
- Transferul de fișiere către un server la distanță.
- Upload multipart/form-data.
- Lucrul cu căi de fișiere în Python.
- Salvarea fișierelor pe server.
- Construirea unui jurnal de transferuri.
- Returnarea răspunsurilor structurate JSON.
- Generarea listelor de fișiere pe server.
- Parsarea și salvarea JSON-ului primit de la server.
- Persistența locală a datelor JSON.
- Construirea unui chat client-server.
- Utilizarea Tkinter pentru GUI.
- Actualizarea dinamică a unei interfețe grafice.
- Gestionarea utilizatorilor conectați.
- Schimbul de mesaje în timp real.
- Verificarea integrității documentelor.
- Generarea hash-urilor MD5.
- Construirea unui registru de veridicitate.
- Validarea unui document pe baza hash-ului.
- Diferența dintre document original și document modificat.
- Execuția la distanță a scripturilor Python prin PHP.
- Utilizarea `shell_exec`, `popen` și `pclose`.
- Transmiterea parametrilor către scripturi Python.
- Controlul aplicațiilor GUI prin HTTP.
- Crearea de popup-uri parametrizabile.
- Utilizarea argumentelor din linia de comandă (`sys.argv`).
- Gestionarea ferestrelor Tkinter.
- Automatizarea închiderii aplicațiilor prin timer.
- Execuția la distanță a executabilelor compilate.
- Compilarea Python în `.exe` cu PyInstaller.
- Diferența dintre script Python și executabil compilat.
- Flux complet:
client Python → HTTP → PHP → execuție aplicație pe server.

Titluri sectiuni:

- Trimiterea de mesaje text pe serverul de la distanta
- Injectie trend
- Upload de fisiere pe serverul de la distanta
- Upload + răspuns structurat + info fișier primit
- Upload + răspuns structurat + listă de fișiere primite
- Chat
- Verificare integritate documente
- Execuție la distanță Python prin PHP
- Execuție la distanta a fișierelor compilate (*.exe)

endpoint HTTP

Este un punct de acces către o funcție/serviciu de pe server.

Ex:

http://192.168.1.5/upload_si_primeste_json/

http://www.domeniu.com/upload_si_primeste_json/

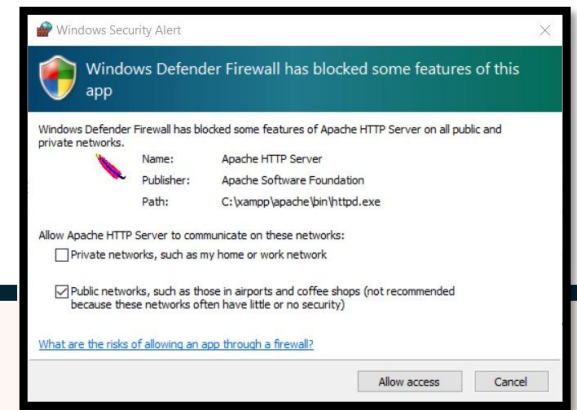
```
ip.py
C: > Users > Paul > Desktop > ip.py > ...
1  import socket
2
3  hostname = socket.gethostname()
4  ip = socket.gethostbyname(hostname)
5
6  print("IP:", ip)
```

Nota: un REST API este un server care oferă funcții prin adrese web.

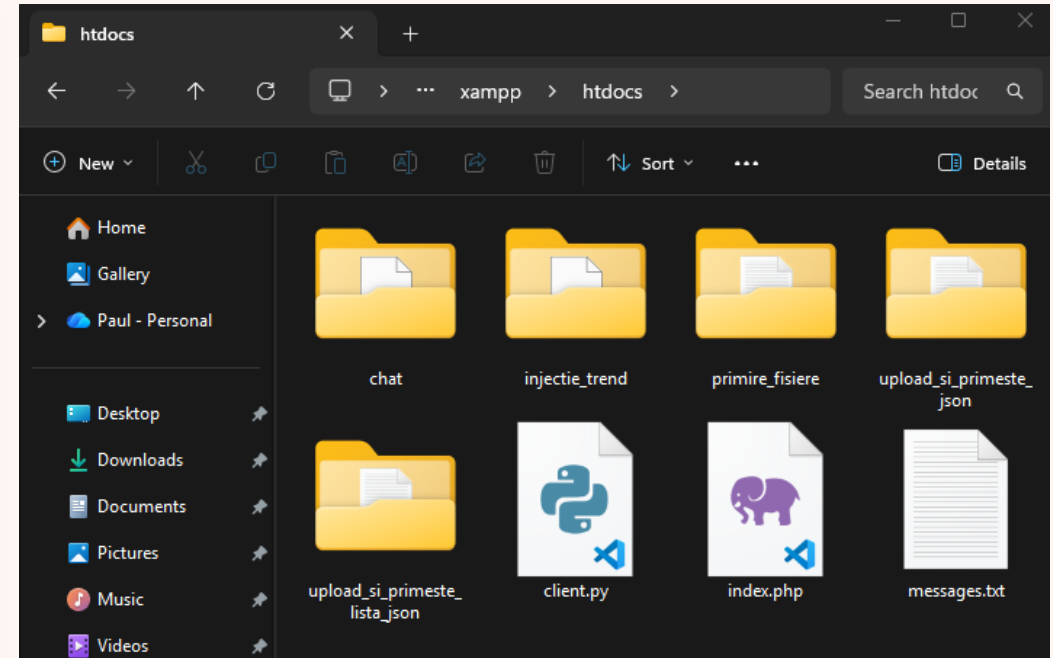
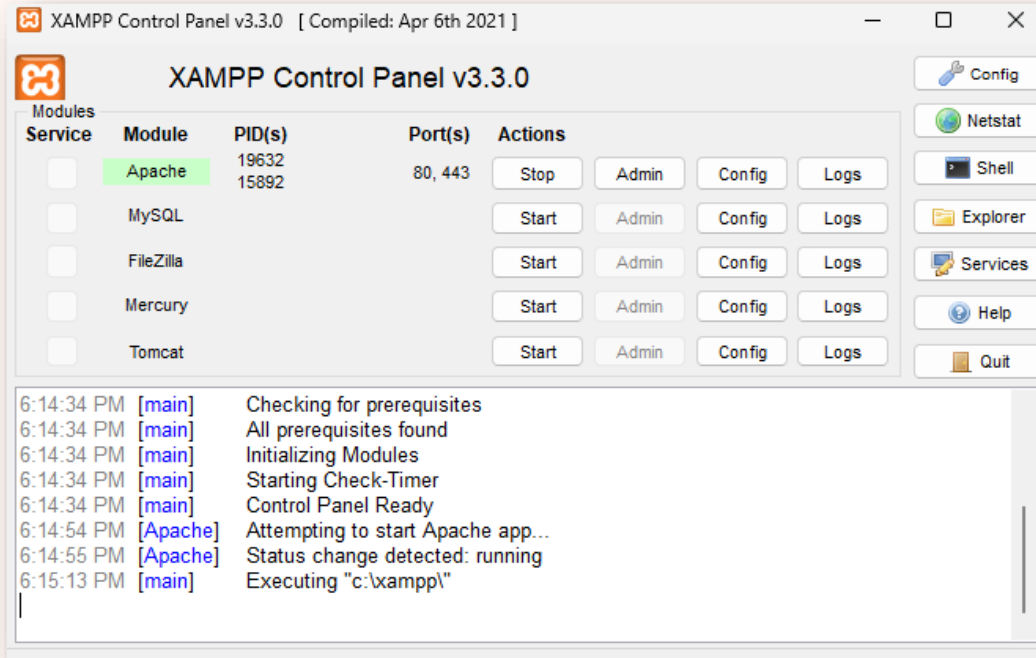
Clientul trimite o cerere HTTP către acea adresă iar serverul răspunde cu date, de obicei JSON.



Server local XAMPP



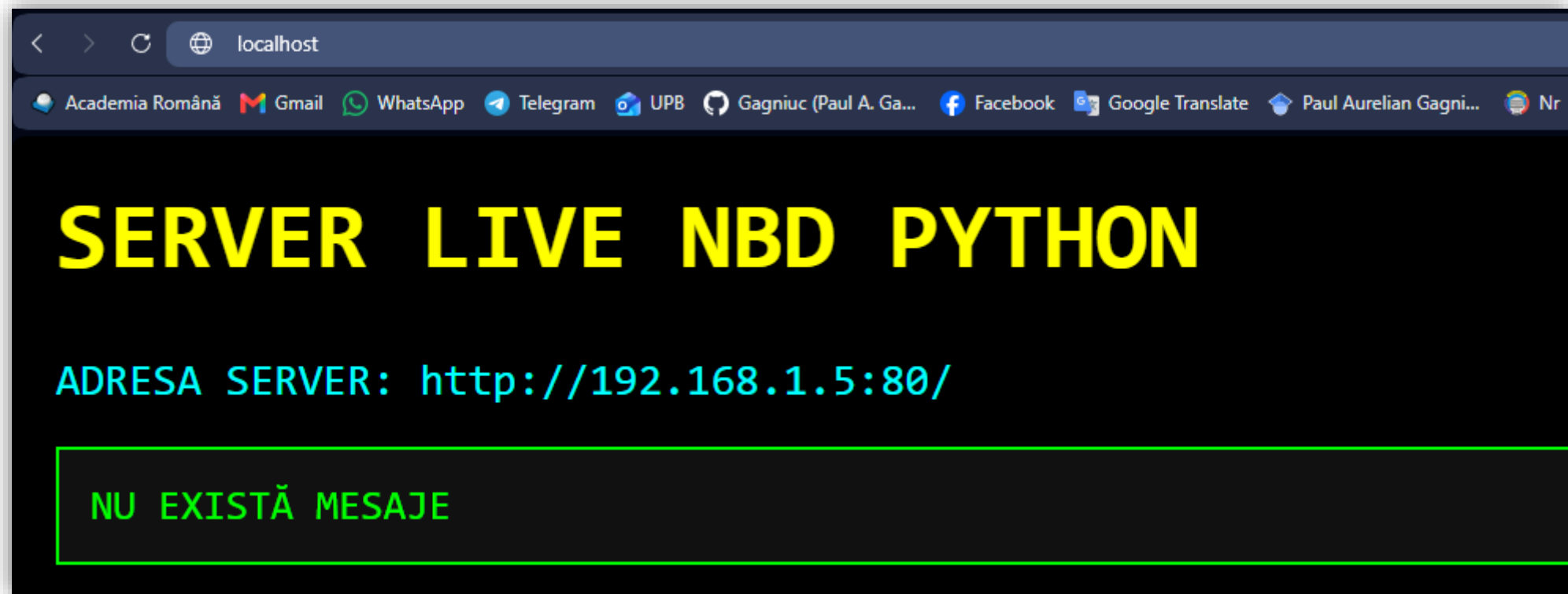
C:\xampp\htdocs



Trimiterea de mesaje text pe serverul de la distanța



Serverul central asteapta mesaje de la client ...



Mesajele se trimit din consola:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\AppData\Local\Programs\Python\Python313\python.exe c:/xampp/htdocs/client.py
Introdu numele: Paul
Scrie mesajul: Salutari colegilor din NBD!
Răspuns server:
MESAJ PRIMIT
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> |
```

Si sunt receptionate de serverul de la distanta:

 localhost

 Academia Română Gmail WhatsApp Telegram UPB Gagniuc (Paul A. Ga... Facebook Google Translate Paul Aurelian Gagni... Nr Nr Acces resurse științif... Academia Tehnica...

SERVER LIVE NBD PYTHON

ADRESA SERVER: <http://192.168.1.5:80/>

17:40:40 | IP CLIENT: 192.168.1.5 | NUME: Paul | MESAJ: Salutari colegilor din NBD!

C: > xampp > htdocs > client.py > ...

```
2  import urllib.request
3
4  adresa_server = "http://192.168.1.5/"
5
6  nume = input("Introdu numele: ")
7  mesaj = input("Scrie mesajul: ")
8
9  date = {}
10
11  date["nume"] = nume
12  date["mesaj"] = mesaj
13
14  date_codificate = urllib.parse.urlencode(date)
15  date_codificate = date_codificate.encode("utf-8")
16
17  cerere = urllib.request.Request(
18      adresa_server,
19      data=date_codificate,
20      method="POST"
21  )
22
23  raspuns = urllib.request.urlopen(cerere)
24
25  continut_raspuns = raspuns.read()
26  continut_raspuns = continut_raspuns.decode("utf-8")
27
28  print("Răspuns server:")
29  print(continut_raspuns)
```

Scrieti partea de client:

Transformă datele Python într-un text compatibil HTTP (ex., nume=Paul&mesaj=Salut), iar a doua transformă acel text în bytes pentru a putea fi trimis prin rețea către server.

Construiește cererea HTTP care va fi trimisă către server, folosind adresa serverului, datele codificate și metoda POST.

Wireshark ! de ce criptari?

The image shows a Wireshark packet capture window. The top pane displays a list of captured packets. Packet 6 is selected, showing an HTTP POST request from 192.168.1.5 to 192.168.1.5. The bottom pane shows the details of this packet, including the raw data and the reassembled TCP segment. The raw data shows the sequence number 3605230040 and the acknowledgment number 3687624. The reassembled TCP segment shows the sequence number 3605230040 and the acknowledgment number 3687624. The packet details pane shows the following information:

- Sequence Number (raw): 3605230040
- [Next Sequence Number: 212 (relative)]
- Acknowledgment Number: 1 (relative)
- Acknowledgment number (raw): 3687624
- 0101 = Header Length: 20 bytes
- Flags: 0x018 (PSH, ACK)
- Window: 255
- [Calculated window size: 65280]
- [Window size scaling factor: 256]
- Checksum: 0xaeabc [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- [Timestamps]
- [SEQ/ACK analysis]
- [Client Contiguous Streams: 1]
- [Server Contiguous Streams: 1]
- TCP payload (26 bytes)

The packet details pane also shows the raw data and the reassembled TCP segment. The raw data shows the sequence number 3605230040 and the acknowledgment number 3687624. The reassembled TCP segment shows the sequence number 3605230040 and the acknowledgment number 3687624. The packet details pane shows the following information:

- Sequence Number (raw): 3605230040
- [Next Sequence Number: 212 (relative)]
- Acknowledgment Number: 1 (relative)
- Acknowledgment number (raw): 3687624
- 0101 = Header Length: 20 bytes
- Flags: 0x018 (PSH, ACK)
- Window: 255
- [Calculated window size: 65280]
- [Window size scaling factor: 256]
- Checksum: 0xaeabc [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- [Timestamps]
- [SEQ/ACK analysis]
- [Client Contiguous Streams: 1]
- [Server Contiguous Streams: 1]
- TCP payload (26 bytes)

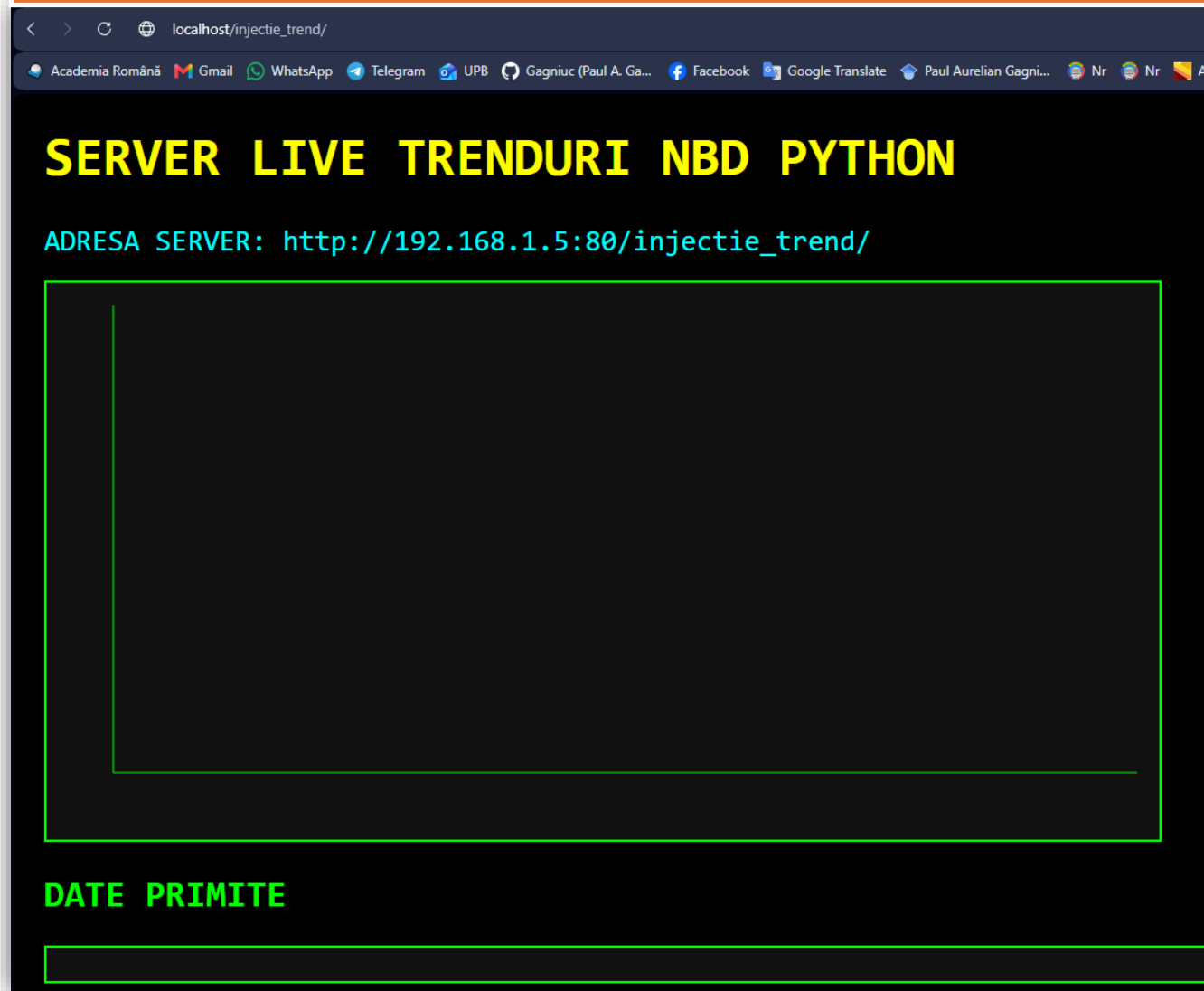
Datele in clar
pot fi captate!

Injectie trend



Injectie de valori

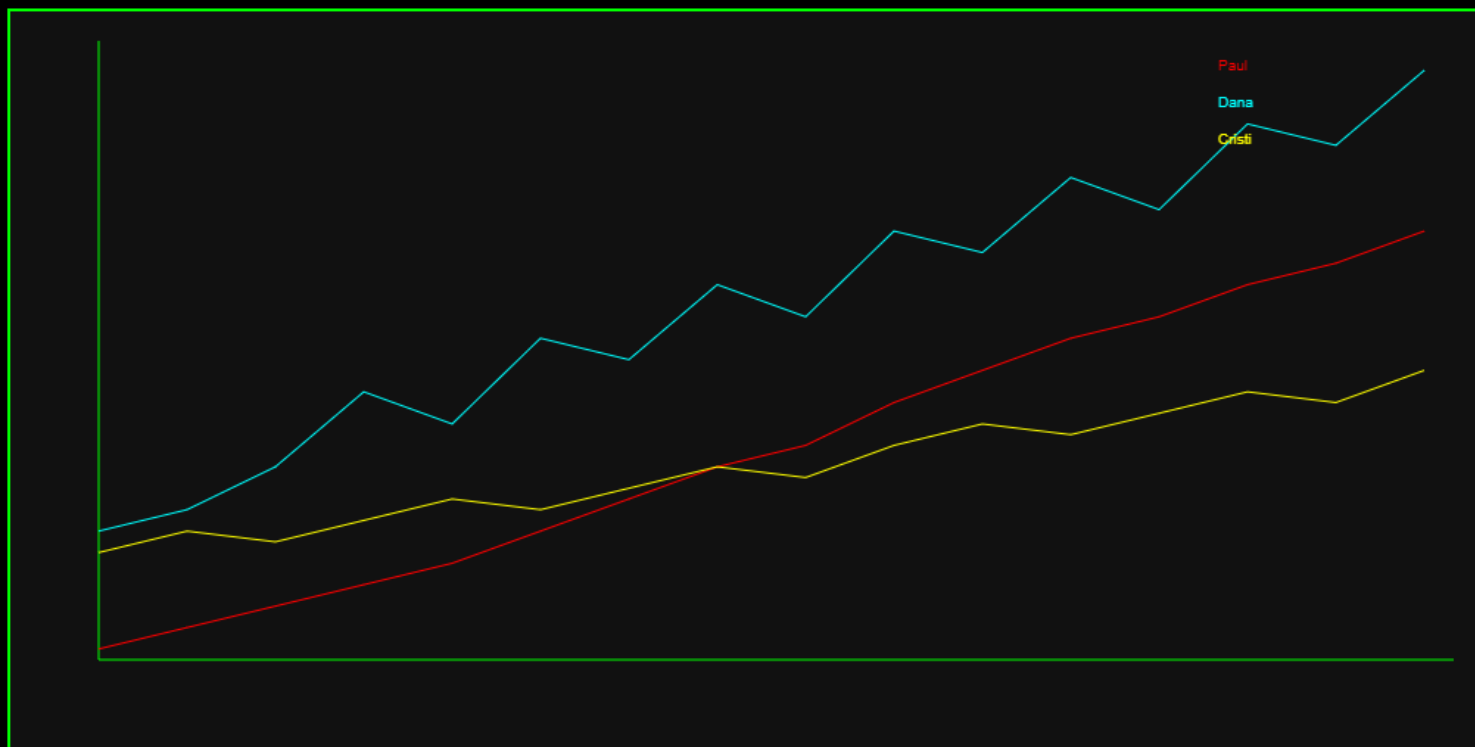
In client.py se schimba doar endpoint-ul



```
client.py X
C: > xampp > htdocs > injectie_trend > client.py > ...
1 import urllib.parse
2 import urllib.request
3
4 adresa_server = "http://192.168.1.5/injectie_trend/"
5
6 nume = input("Introdu numele: ")
7 mesaj = input("Scrie mesajul: ")
8
9 date = {}
10
11 date["nume"] = nume
12 date["mesaj"] = mesaj
13
14 date_codificate = urllib.parse.urlencode(date)
15 date_codificate = date_codificate.encode("utf-8")
16
17 cerere = urllib.request.Request(
18     adresa_server,
19     data=date_codificate,
20     method="POST"
21 )
22
23 raspuns = urllib.request.urlopen(cerere)
24
25 continut_raspuns = raspuns.read()
26 continut_raspuns = continut_raspuns.decode("utf-8")
27
28 print("Răspuns server:")
29 print(continut_raspuns)
```

SERVER LIVE TRENDURI NBD PYTHON

ADRESA SERVER: http://192.168.1.5:80/injectie_trend/



Rezultat !

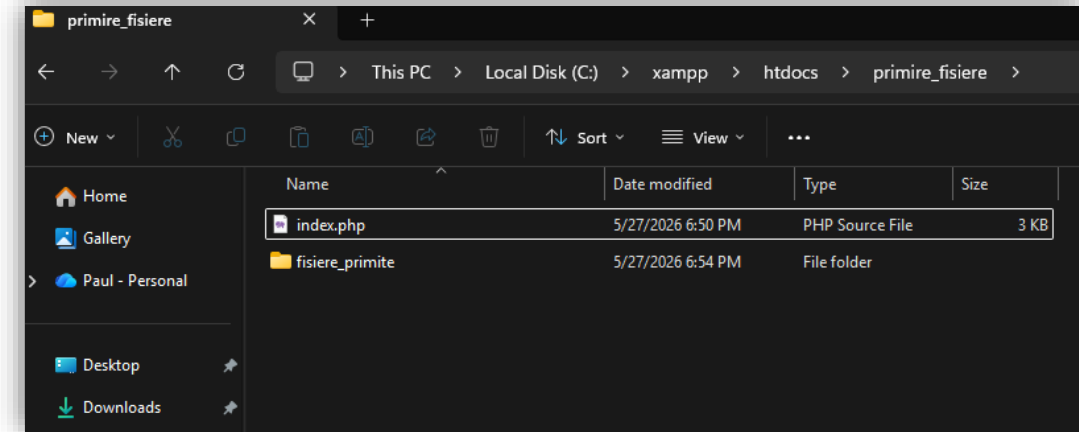
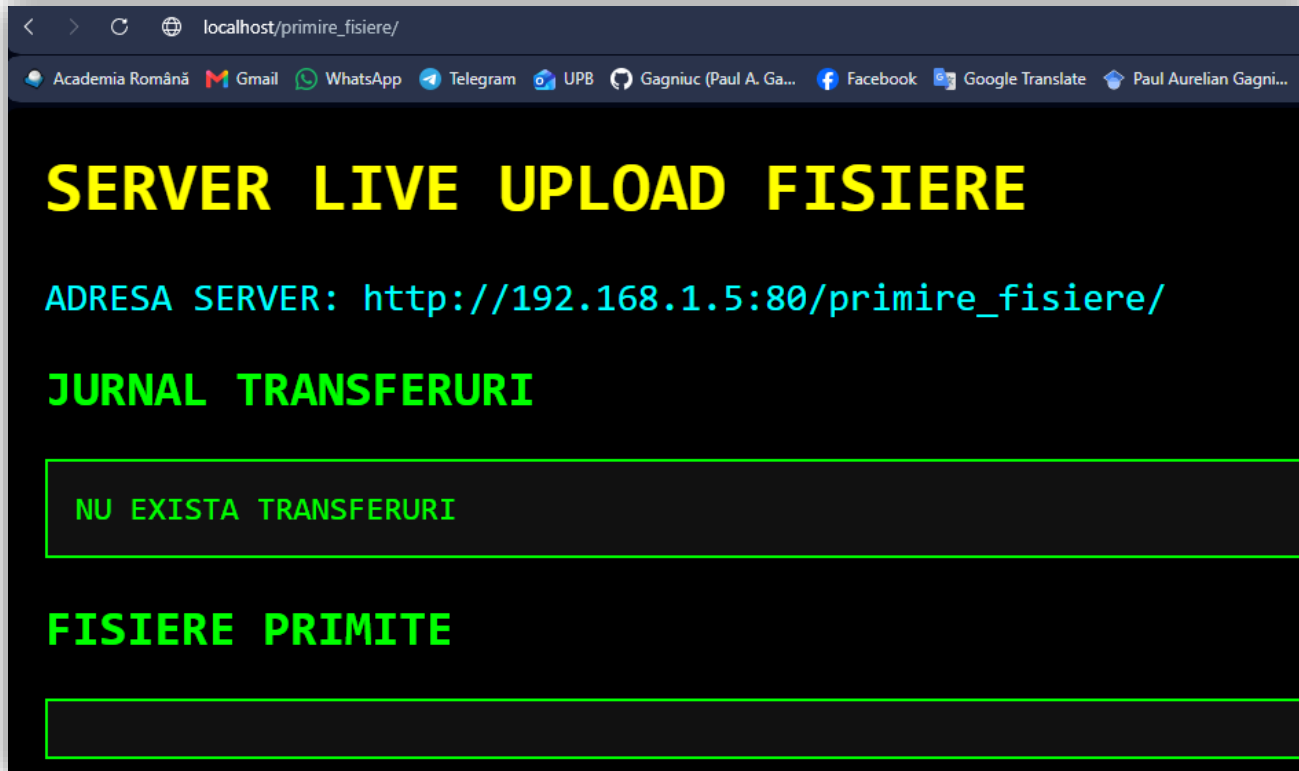
DATE PRIMATE

20:56:22	IP CLIENT: 192.168.1.5	NUME: Paul	VALORI: 1,3,5,7,9,12,15,18,20,24,27,30,32,35,37,40
20:57:10	IP CLIENT: 192.168.1.5	NUME: Dana	VALORI: 12,14,18,25,22,30,28,35,32,40,38,45,42,50,48,55
20:57:44	IP CLIENT: 192.168.1.5	NUME: Cristi	VALORI: 10,12,11,13,15,14,16,18,17,20,22,21,23,25,24,27

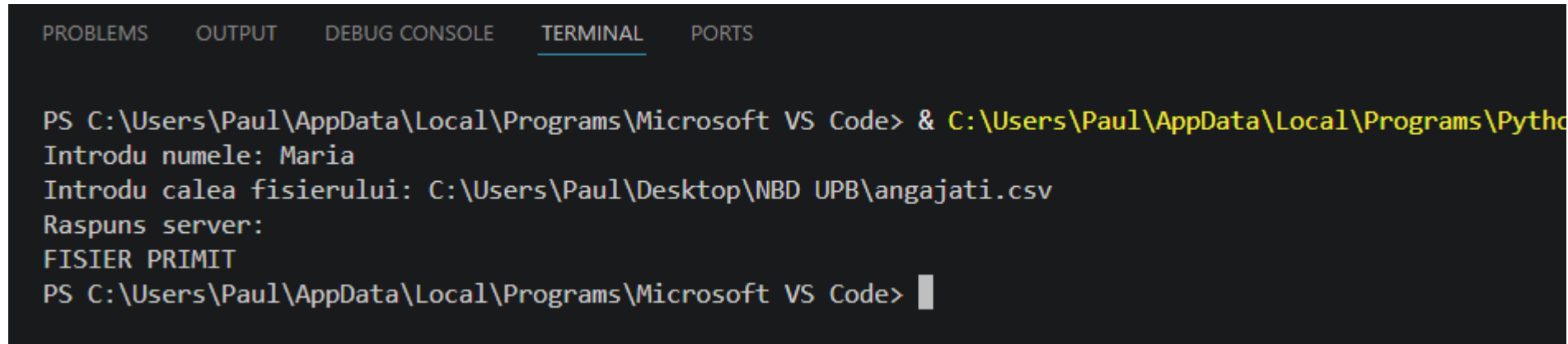
Upload de fisiere pe serverul de la distanta



Upload de fisiere pe serverul de la distanta:



Transferul se face din consola:



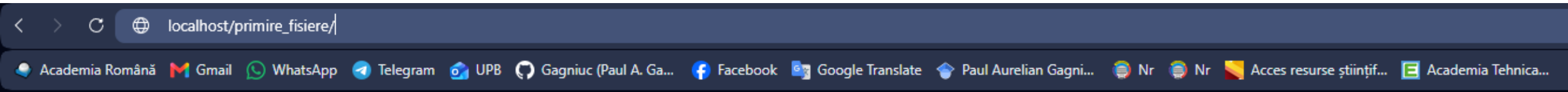
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\AppData\Local\Programs\Python\Python38-32\python.exe C:\Users\Paul\AppData\Local\Programs\Python\Python38-32\Scripts\python.exe C:\Users\Paul\AppData\Local\Programs\Python\Python38-32\Scripts\python.exe C:\Users\Paul\AppData\Local\Programs\Python\Python38-32\Scripts\python.exe
Introdu numele: Maria
Introdu calea fisierului: C:\Users\Paul\Desktop\NBD UPB\angajati.csv
Raspuns server:
FISIER PRIMIT
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```

Imputul este calea catre fisier:

C:\Users\Paul\Desktop\NBD UPB\angajati.csv

Dupa primirea fisierului:



SERVER LIVE UPLOAD FISIERE

ADRESA SERVER: `http://192.168.1.5:80/primire_fisiere/`

JURNAL TRANSFERURI

```
2026-05-27 18:05:56 | IP CLIENT: 192.168.1.5 | STUDENT: Paul | FISIER: angajati.csv
```

FISIERE PRIMITE

```
angajati.csv
```

Scrieti partea de client pentru trimiterea de fisiere:

boundary este textul special care separă în pachetul HTTP câmpul nume_student de câmpul fisier și marchează unde începe și unde se termină fiecare parte trimisă către server.

codul simulează uploadul unui formular HTML din browser, cu un câmp text și un fișier atașat.

\r = carriage return (mută cursorul la începutul liniei)
\n = newline (trece pe linia următoare)

C: > xampp > httdocs > primire_fisiere > client.py > ...

```
1 import urllib.request
2
3 adresa_server = "http://192.168.1.5/primire_fisiere/"
4
5 nume_student = input("Introdu numele: ")
6 cale_fisier = input("Introdu calea fisierului: ")
7
8 fisier = open(cale_fisier, "rb")
9 continut_fisier = fisier.read()
10 fisier.close()
11
12 nume_fisier = cale_fisier.split("\\")[1]
13
14 boundary = "-----NBDPythonBoundary" }
15
16 date = b""
17
18 date += ("--" + boundary + "\r\n").encode("utf-8")
19 date += b'Content-Disposition: form-data; name="nume_student"\r\n\r\n'
20 date += nume_student.encode("utf-8")
21 date += b"\r\n"
22
23 date += ("--" + boundary + "\r\n").encode("utf-8")
24 date += ('Content-Disposition: form-data; name="fisier"; filename="' + nume_fisier + '"\r\n').encode("utf-8")
25 date += b'Content-Type: application/octet-stream\r\n\r\n'
26
27 date += continut_fisier
28 date += b"\r\n"
29
30 date += ("--" + boundary + "--\r\n").encode("utf-8")
31
32 cerere = urllib.request.Request(adresa_server)
33
34 cerere.add_header(
35     "Content-Type",
36     "multipart/form-data; boundary=" + boundary
37 )
38
39 cerere.data = date
40
41 raspuns = urllib.request.urlopen(cerere)
42
43 continut_raspuns = raspuns.read()
44 continut_raspuns = continut_raspuns.decode("utf-8")
45
46 print("Raspuns server:")
47 print(continut_raspuns)
```

Upload + răspuns
structurat + info
fișier primit.



Exercitiu! Ne mutam pe endpoint-ul:

http://192.168.1.5/upload_si_primeste_json/

```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\
Introdu numele: Paul
Introdu calea fisierului: C:\Users\Paul\Desktop\NBD UPB\angajati.csv
Raspuns server:
{
  "status": "ok",
  "mesaj": "Fisier primit",
  "student": "Paul",
  "ip_client": "192.168.1.5",
  "ora": "2026-05-27 19:14:25",
  "fisier_primit": "angajati.csv",
  "fisiere_primate": [
    "angajati.csv"
  ]
}
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> █
```

- 1) Folosim clientul anterior, insa,
- 2) schimbam link-ul din client, la care se face cererea (link-ul cu rosu).
- 3) Apoi, dupa incarcarea unui fisier la noul endpoint, serverul returneaza lista de fisiere existenta sub forma de JSON.

REST API = un set de adrese web prin care un program vorbește cu un server.

Modificati client.py prin adaugarea modulului de salvare a raspunsului in fisier JSON

```
import os

# Clientul poate salva raspunsul serverului ca fisier JSON local.

cale_curenta = os.path.dirname(os.path.abspath(__file__))

cale_json = os.path.join(cale_curenta, "raspuns_server.json")

fisier_json = open(cale_json, "w", encoding="utf-8")
fisier_json.write(continut_raspuns)
fisier_json.close()
```

raspuns_server.json



```
{
  "status": "ok",
  "mesaj": "Fisier primit",
  "student": "Dana",
  "ip_client": "192.168.1.5",
  "ora": "2026-05-27 20:18:06",
  "fisier_primit": "pisica.jpg",
  "fisiere_primite": [
    "angajati.csv",
    "pisica.jpg"
  ]
}
```

Upload + răspuns
structurat + listă de
fișiere primite.

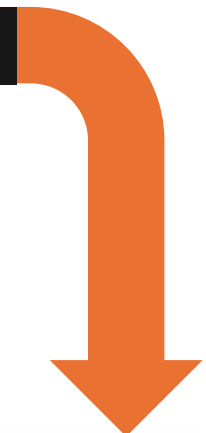


Ne mutam pe: upload_si_primeste_lista_json

http://192.168.1.5/upload_si_primeste_lista_json/

```
adresa_server = "http://192.168.1.5/upload_si_primeste_lista_json/"
```

Acelasi client.py ne va salva lista fisierelor incarcate pe server.



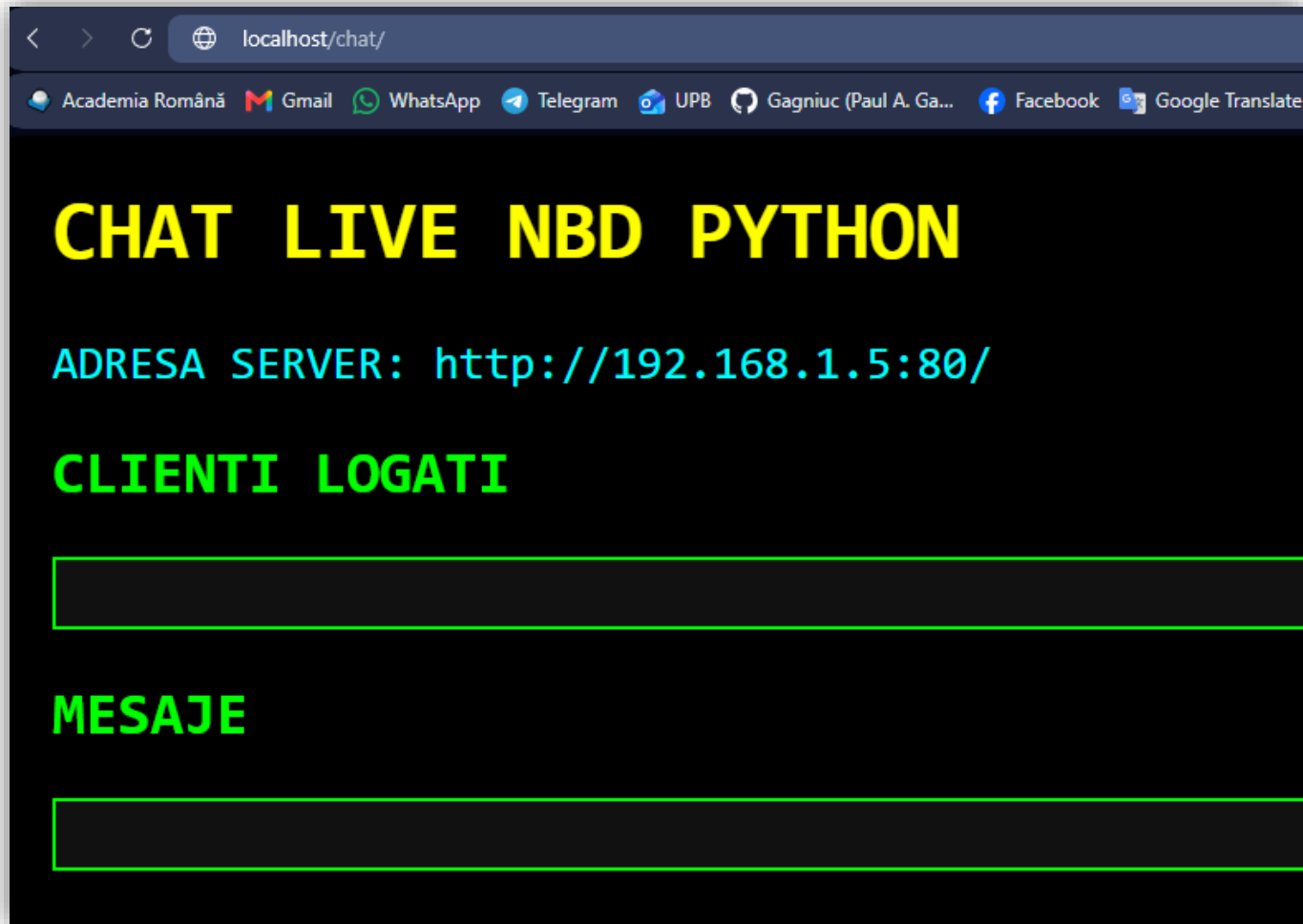
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Introdu numele: Gelu
Introdu calea fisierului: C:\Users\Paul\Desktop\NBD UPB\pisica.jpg
Raspuns server:
[
    "angajati.csv",
    "pisica.jpg"
]
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```

```
client.py
C: > xampp > htdocs > upload_si_primeste_lista_json > client.py > ...
1  import urllib.request
2  import os
3
4  adresa_server = "http://192.168.1.5/upload_si_primeste_lista_json/"
5
6  nume_student = input("Introdu numele: ")
7  cale_fisier = input("Introdu calea fisierului: ")
8
```



Server



```
index.php client.py X
C: > xampp > htdocs > chat > client.py > ...
1 import urllib.parse
2 import urllib.request
3 import tkinter as tk
4
5 adresa_server = "http://192.168.1.5/chat/"
6
7 def trimite_la_server(actiune, mesaj):
8     date = {}
9     date["actiune"] = actiune
10    date["nume"] = camp_nume.get()
11    date["mesaj"] = mesaj
12
13    date = urllib.parse.urlencode(date).encode("utf-8")
14
15    cerere = urllib.request.Request(
16        adresa_server,
17        data=date,
18        method="POST"
19    )
20
21    raspuns = urllib.request.urlopen(cerere)
22
23    return raspuns.read().decode("utf-8")
24
25 def afiseaza_chat(text):
26     zona_chat.delete("1.0", tk.END)
27     zona_chat.insert(tk.END, text)
28
29 def trimite_mesaj():
30     mesaj = camp_mesaj.get()
31
32     if mesaj == "":
33         return
34
35     text = trimite_la_server("mesaj", mesaj)
36
37     afiseaza_chat(text)
38
39     camp_mesaj.delete(0, tk.END)
40
41 def actualizeaza_chat():
42     text = trimite_la_server("prezenta", "")
43     afiseaza_chat(text)
44
```

trimite_la_server(actiune, mesaj) ia acțiunea, numele și mesajul, le transformă în date HTTP, le trimite prin POST la serverul PHP și întoarce textul primit ca răspuns.

urlencode transformă caracterele speciale în forma URL encoded, de exemplu:

Salut Paul! → Salut+Paul%21

Pentru fișiere binare se folosește de obicei multipart/form-data, nu urlencode.

afiseaza_chat(text)

Șterge zona de chat și afișează textul nou primit de la server.

trimite_mesaj()

la mesajul introdus de utilizator, îl trimite la server și actualizează chatul.

actualizeaza_chat()

Cere periodic ultimele mesaje de la server și actualizează fereastra chatului.

```

45 fereastra = tk.Tk()
46
47 fereastra.title("Chat NBD Python")
48
49 fereastra.geometry("800x600")
50
51 tk.Label(
52     fereastra,
53     text="Nume:"
54 ).pack()
55
56 camp_nume = tk.Entry(
57     fereastra,
58     font=("Arial", 14)
59 )
60
61 camp_nume.pack(
62     fill="x",
63     padx=10
64 )
65
66 zona_chat = tk.Text(
67     fereastra,
68     font=("Consolas", 13)
69 )
70
71 zona_chat.pack(
72     fill="both",
73     expand=True,
74     padx=10,
75     pady=10
76 )
77
78 tk.Label(
79     fereastra,
80     text="Mesaj:"
81 ).pack()
82
83 camp_mesaj = tk.Entry(
84     fereastra,
85     font=("Arial", 14)
86 )

```

fereastra = tk.Tk()

Creează fereastra principală a aplicației.

fereastra.title("Chat NBD Python")

Setează titlul ferestrei.

fereastra.geometry("800x600")

Setează dimensiunea ferestrei la 800×600.

tk.Label(... text="Nume:").pack()

Afișează eticheta „Nume:”.

camp_nume = tk.Entry(...)

Creează câmpul unde utilizatorul introduce numele.

camp_nume.pack(...)

Afișează câmpul de nume în fereastră.

zona_chat = tk.Text(...)

Creează zona unde apar mesajele chatului.

zona_chat.pack(...)

Afișează zona de chat și o extinde în fereastră.

tk.Label(... text="Mesaj:").pack()

Afișează eticheta „Mesaj:”.

camp_mesaj = tk.Entry(...)

Creează câmpul unde utilizatorul scrie mesajul.

```

88     camp_mesaj.pack(
89         fill="x",
90         padx=10
91     )
92
93     camp_mesaj.bind(
94         "<Return>",
95         lambda event: trimite_mesaj()
96     )
97
98     cadru_butoane = tk.Frame(fereastra)
99
100    cadru_butoane.pack(pady=10)
101
102    buton_trimite = tk.Button(
103        cadru_butoane,
104        text="TRIMITE",
105        font=("Arial", 14),
106        width=15,
107        command=trimite_mesaj
108    )
109
110    buton_trimite.pack(
111        side="left",
112        padx=10
113    )
114
115    buton_actualizare = tk.Button(
116        cadru_butoane,
117        text="ACTUALIZEAZA CHAT",
118        font=("Arial", 14),
119        width=20,
120        command=actualizeaza_chat
121    )
122
123    buton_actualizare.pack(
124        side="left",
125        padx=10
126    )
127
128    fereastra.mainloop()

```

camp_mesaj.pack(...)

Afișează câmpul pentru scrierea mesajului.

camp_mesaj.bind("<Return>", ...)

Trimite mesajul când utilizatorul apasă Enter.

cadru_butoane = tk.Frame(fereastra)

Creează un container pentru butoane.

cadru_butoane.pack(...)

Afișează containerul butoanelor.

buton_trimite = tk.Button(...)

Creează butonul „TRIMITE”.

command=trimite_mesaj

Leagă butonul de funcția care trimite mesajul.

buton_trimite.pack(...)

Afișează butonul în fereastră.

buton_actualizare = tk.Button(...)

Creează butonul „ACTUALIZEAZA CHAT”.

command=actualizeaza_chat

Leagă butonul de funcția care cere ultimele mesaje de la server.

buton_actualizare.pack(...)

Afișează butonul în fereastră.

fereastra.mainloop()

Pornește bucla principală a interfeței grafice și menține aplicația deschisă.

CHAT LIVE NBD PYTHON

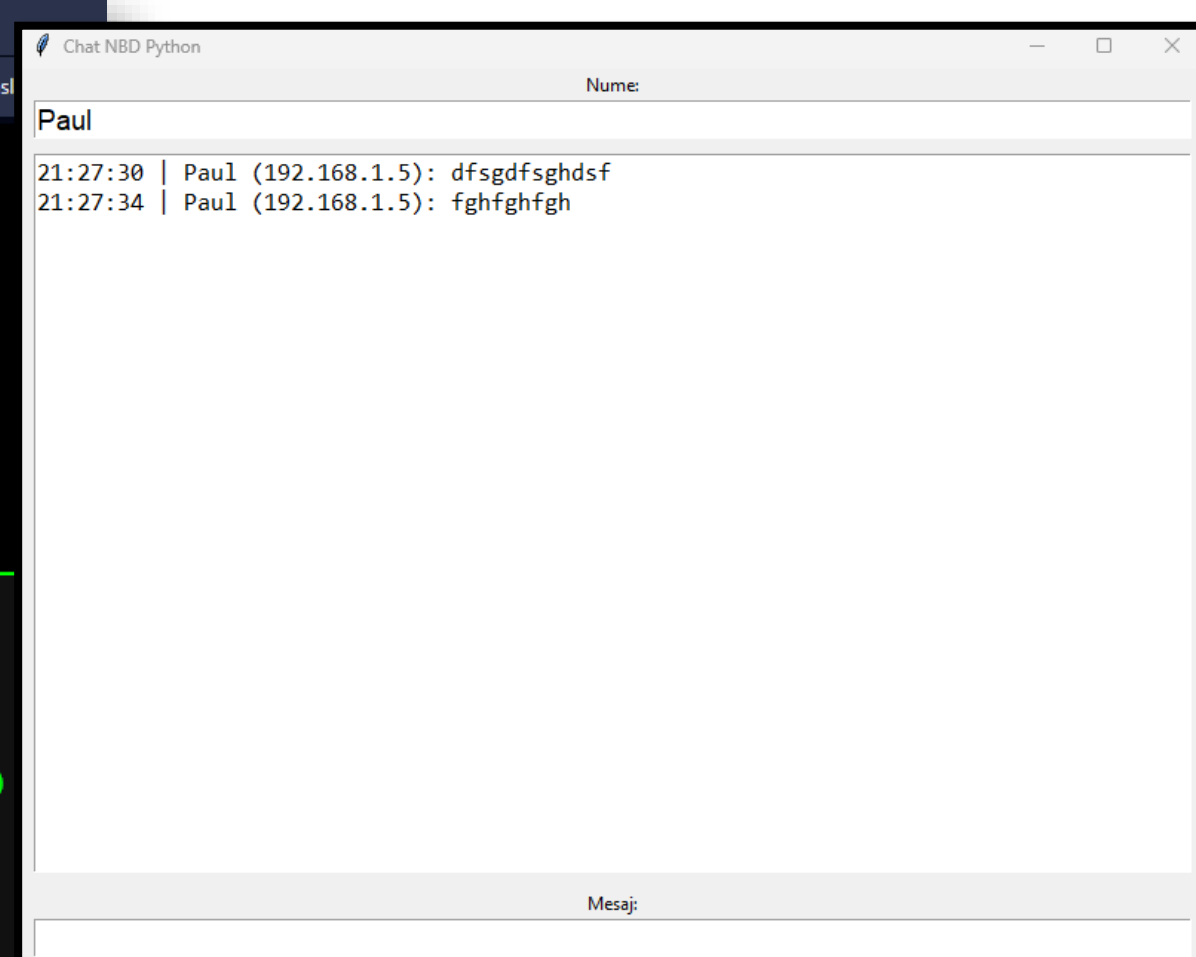
ADRESA SERVER: <http://192.168.1.5:80/chat/>

CLIENTI LOGATI

```
P | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:13:50
Paul | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:27:34
Crist | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:20:28
Cristi | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:21:00
asdas | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:23:11
Pu | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:23:17
S | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:23:21
Simona | IP: 192.168.1.5 | ULTIMA ACTIVITATE: 21:24:19
```

MESAJE

```
21:27:30 | Paul (192.168.1.5): dfsgdfsgghdsf
21:27:34 | Paul (192.168.1.5): fghfghfgh
```



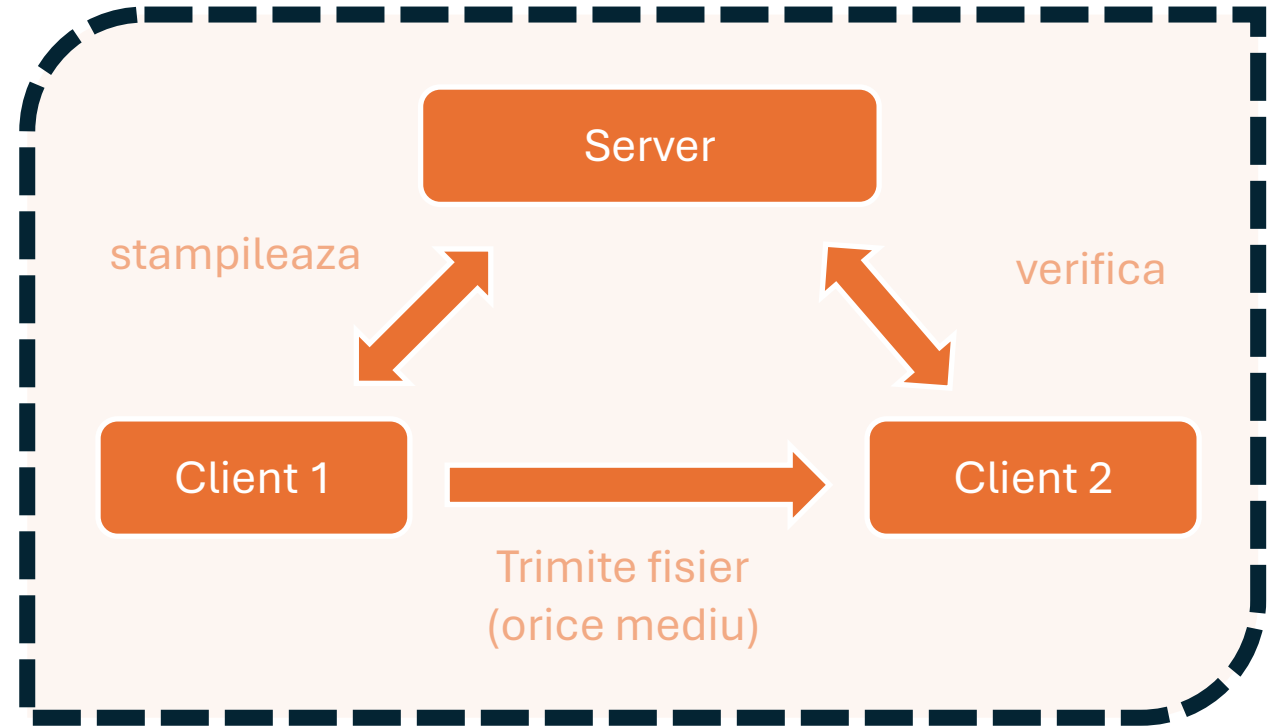
Testare:

Clienti - Server

Verificare integritate documente



Server care verifică hash-uri documente



Stampilare fisier

C:\Users\Paul\Desktop\NBD UPB\angajati.csv

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Us
Actiune stampileaza (s)/verifica (v): s
Calea fisierului: C:\Users\Paul\Desktop\NBD UPB\angajati.csv
{
  "status": "registered",
  "mesaj": "Document inregistrat",
  "document": "angajati.csv",
  "md5": "959ee8abe6518e553ef9f1ed732a9284"
}
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> █
```

localhost/md5/

Academia Română Gmail WhatsApp Telegram UPB Gagniuc (Paul A. Ga... Facebook Google Translate Paul Aurelian Gagni... Nr Nr Acces resurse științ

REGISTRU VERIDICITATE DOCUMENTE

ADRESA SERVER: <http://192.168.1.5:80/md5/>

DOCUMENTE INREGISTRATE

2026-05-27 22:43:02		angajati.csv		959ee8abe6518e553ef9f1ed732a9284		IP: 192.168.1.5
---------------------	--	--------------	--	----------------------------------	--	-----------------

Verificare fisier primit

C:\Users\Paul\Desktop\NBD UPB\angajati.csv

```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\bin\node.exe .\node_modules\actiune-stampileaza\index.js --verifica C:\Users\Paul\Desktop\NBD UPB\angajati.csv
Actiune stampileaza (s)/verifica (v): v
Calea fisierului: C:\Users\Paul\Desktop\NBD UPB\angajati.csv
{
  "status": "valid",
  "mesaj": "Documentul exista in registru",
  "document_original": "angajati.csv",
  "md5": "959ee8abe6518e553ef9f1ed732a9284"
}
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```



C: > xampp > htdocs > md5 > client.py > ...

```
1  import hashlib
2  import urllib.parse
3  import urllib.request
4  import os
5
6  adresa_server = "http://192.168.1.5/md5/"
7
8  actiune = input("Actiune stampileaza (s)/verifica (v): ")
9  cale_fisier = input("Calea fisierului: ")
10
11  fisier = open(cale_fisier, "rb")
12  continut = fisier.read()
13  fisier.close()
14
15  md5 = hashlib.md5(continut).hexdigest()
16  nume_document = os.path.basename(cale_fisier)
17
18  date = {}
19  date["actiune"] = actiune
20  date["nume_document"] = nume_document
21  date["md5"] = md5
22
23  date = urllib.parse.urlencode(date).encode("utf-8")
24
25  cerere = urllib.request.Request(adresa_server, data=date, method="POST")
26  raspuns = urllib.request.urlopen(cerere)
27
28  text = raspuns.read().decode("utf-8")
29
30  print(text)
```

client.py



Execuție la distanță Python prin PHP



C: > xampp > htdocs > executie > client.py > ...

```
1  import urllib.parse
2  import urllib.request
3
4  adresa_server = "http://192.168.1.5/executie/"
5
6  nume_student = input("Nume student: ")
7
8  print("1 - deschide fereastra rosie")
9  print("2 - deschide fereastra verde")
10 print("3 - deschide fereastra albastra")
11
12 actiune = input("Alege actiunea: ")
13
14 date = {}
15 date["actiune"] = actiune
16 date["student"] = nume_student
17
18 date_codificate = urllib.parse.urlencode(date)
19 date_codificate = date_codificate.encode("utf-8")
20
21 cerere = urllib.request.Request(
22     adresa_server,
23     data=date_codificate,
24     method="POST"
25 )
26
27 raspuns = urllib.request.urlopen(cerere)
28
29 continut = raspuns.read()
30 continut = continut.decode("utf-8")
31
32 print("Raspuns server:")
33 print(continut)
```

client.py



C: > xampp > htdocs > executie > index.php

```
1  <?php
2
3  $actiune = $_POST["actiune"];
4  $student = $_POST["student"];
5
6  if ($actiune == "1") {
7
8      pclose(popen('start /B pythonw fereastra.py "ALERTA 1" red 1 "' . $student . '"', "r"));
9
10     echo "POPUP 1 EXECUTAT";
11 }
12
13 if ($actiune == "2") {
14
15     pclose(popen('start /B pythonw fereastra.py "ALERTA 2" green 2 "' . $student . '"', "r"));
16
17     echo "POPUP 2 EXECUTAT";
18 }
19
20 if ($actiune == "3") {
21
22     pclose(popen('start /B pythonw fereastra.py "ALERTA 3" blue 3 "' . $student . '"', "r"));
23
24     echo "POPUP 3 EXECUTAT";
25 }
26
27 ?>
```

index.php

```
index.php client.py fereastra.py •
C: > xampp > htdocs > executie > fereastra.py > ...
1 import sys
2 import tkinter as tk
3
4 titlu = "FEREASTRA"
5 culoare = "red"
6 numar = "0"
7 student = "necunoscute"
8
9 if len(sys.argv) > 1: titlu = sys.argv[1]
10 if len(sys.argv) > 2: culoare = sys.argv[2]
11 if len(sys.argv) > 3: numar = sys.argv[3]
12 if len(sys.argv) > 4: student = sys.argv[4]
13
14 secunde_ramase = 10
15
16 def actualizeaza_timer():
17     global secunde_ramase
18     eticheta_timer.config(text="Se inchide in " + str(secunde_ramase) + " secunde")
19
20     if secunde_ramase == 0:
21         fereastra.destroy()
22         return
23
24     secunde_ramase = secunde_ramase - 1
25     fereastra.after(1000, actualizeaza_timer)
26
27 fereastra = tk.Tk()
28 fereastra.title(titlu)
29 fereastra.geometry("700x400")
30 fereastra.configure(bg=culoare)
31 fereastra.attributes("-topmost", True)
32 fereastra.lift()
33 fereastra.focus_force()
34
35 eticheta_text = tk.Label(fereastra, text="FEREASTRA " + numar, font=("Arial", 32), bg=culoare, fg="white")
36 eticheta_text.pack(pady=10)
37
38 eticheta_student = tk.Label(fereastra, text="STUDENT: " + student, font=("Arial", 24), bg=culoare, fg="yellow")
39 eticheta_student.pack(pady=5)
40
41 eticheta_timer = tk.Label(fereastra, text="", font=("Arial", 18), bg=culoare, fg="white")
42 eticheta_timer.pack(pady=5)
43
44 actualizeaza_timer()
45
46 fereastra.mainloop()
```

fereastra.py

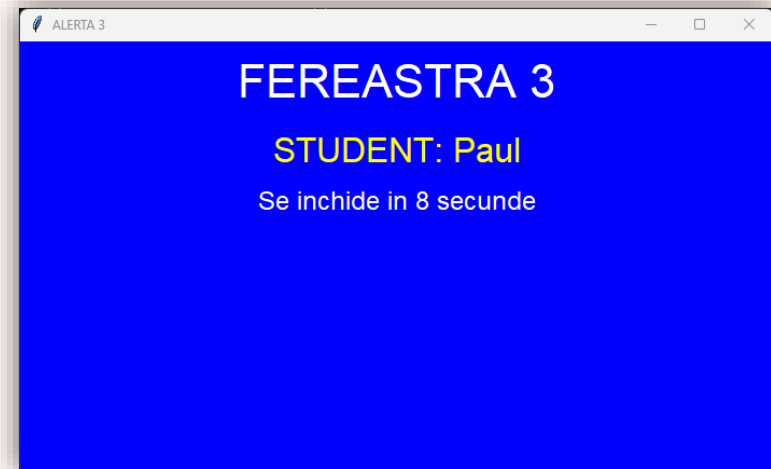
Script popup de
executat pe
server, la cerere!

Testare



```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Us
Nume student: Paul
1 - deschide fereastră rosie
2 - deschide fereastră verde
3 - deschide fereastră albastră
Alege acțiunea: 3
Raspuns server:
POPUP 3 EXECUTAT
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> █
```

client.py



fereastră.py

Execuție la distanță a fișierelor compilate (* .exe)

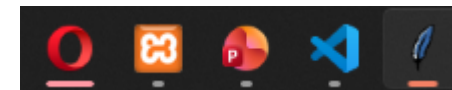
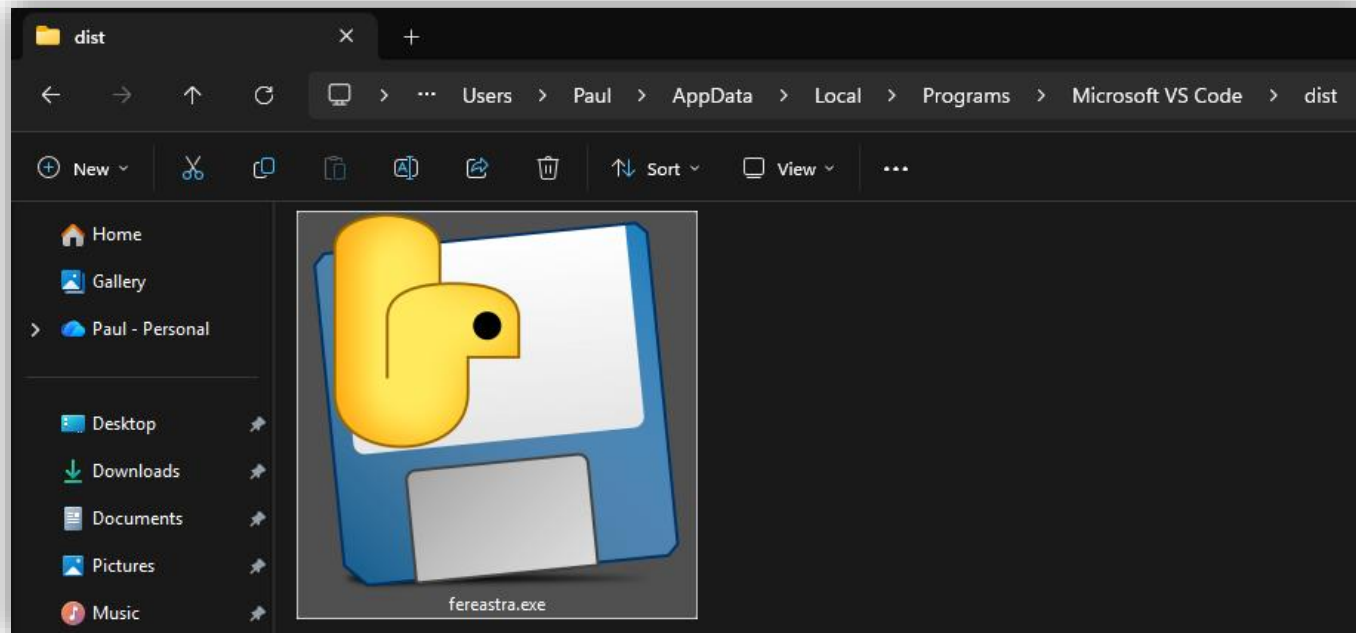


Compilare

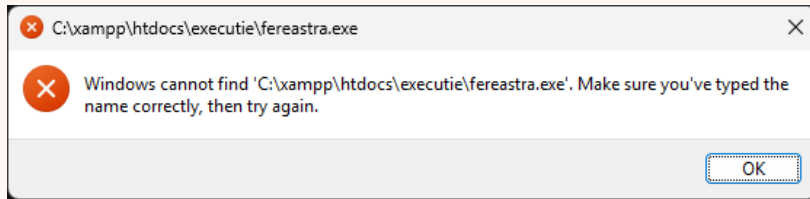
```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> pyinstaller --onefile --windowed c:/xampp/htdocs/exe/fereastra.py
160 INFO: PyInstaller: 6.20.0, contrib hooks: 2026.5
160 INFO: Python: 3.13.7
193 INFO: Platform: Windows-11-10.0.26100-SP0
194 INFO: Python environment: C:\Users\Paul\AppData\Local\Programs\Python\Python313
195 INFO: wrote C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\fereastra.spec
206 INFO: Module search paths (PYTHONPATH):
['C:\\Users\\Paul\\AppData\\Local\\Programs\\Python\\Python313\\Scripts\\pyinstaller.exe',
'C:\\Users\\Paul\\AppData\\Local\\Programs\\Python\\Python313\\python313.zip',
'C:\\Users\\Paul\\AppData\\Local\\Programs\\Python\\Python313\\DLLs',
```

pip install pyinstaller

pyinstaller --onefile --windowed c:/xampp/htdocs/exe/fereastra.py

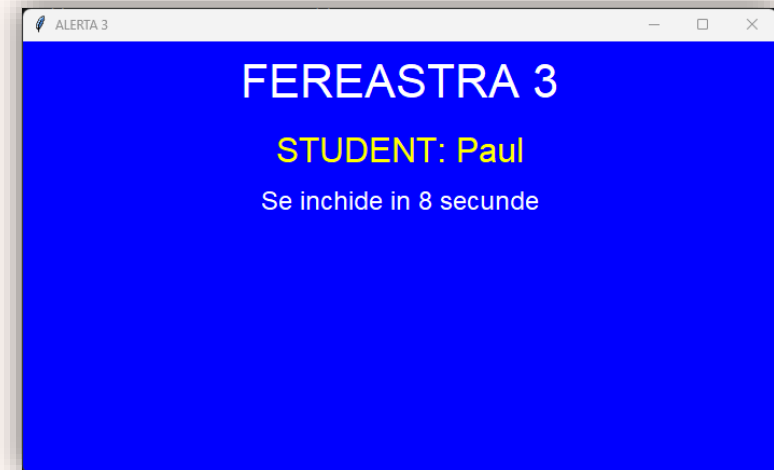


Testare este la fel ca cea precedenta



```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
Nume student: Paul
1 - deschide fereastră roșie
2 - deschide fereastră verde
3 - deschide fereastră albastră
Alege acțiunea: 3
Răspuns server:
POPOP 3 EXECUTAT
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> 
```

client.py



fereastră.exe

Ză end!

